ProSpy: Fake Social Media Account Detection Using Neural Networks

Technical Documentation

Author: Archi Sharma
B.Tech in Artificial Intelligence & Machine Learning

Date: July 2026

Table of Contents

Abstract

The proliferation of fake social media accounts poses significant risks to platform integrity, user trust, and societal discourse. ProSpy addresses this challenge through a deep learning-based binary classification system utilizing an Artificial Neural Network (ANN) implemented in TensorFlow/Keras. The model analyzes profile metadata to distinguish genuine accounts from fake ones with high accuracy.

Key contributions include comprehensive data preprocessing (handling imbalance via balancing techniques, feature selection, and StandardScaler normalization), a multi-layer perceptron architecture with ReLU activations and dropout regularization, and deployment-ready artifacts including a .keras model and preprocessing pickles. Evaluation on train/validation/test splits yields accuracies of approximately 95.69% (train), 93.10% (validation), and 91.53% (test), with strong F1-scores exceeding 91%.

The system integrates with a full-stack web application (React frontend + Flask backend), enabling real-time predictions. This report details the end-to-end development lifecycle, positioning ProSpy as a scalable solution for social media moderation and user safety tools. Future enhancements target explainable AI and graph-based modeling for improved generalization.

---

1. Introduction

1.1 Problem Statement

Fake accounts on platforms like Instagram, Facebook, and Twitter (X) are used for spam, misinformation, bot-driven amplification, identity theft, and harassment. Traditional rule-based detection struggles with sophisticated fakes that mimic human behavior. ProSpy leverages machine learning on structured profile features to provide an automated, data-driven detection mechanism.

1.2 Importance of Fake Social Media Account Detection

- **Security:** Prevents scams and phishing.
- **Platform Health:** Maintains authentic engagement metrics.
- **Societal Impact:** Reduces spread of disinformation.
- **Business Value:** Enhances ad targeting and user experience.

Empirical studies show fake accounts can comprise 10–40% of users on major platforms, underscoring the urgency.

1.3 Objectives of ProSpy

- Develop a high-accuracy ANN classifier.
- Implement robust preprocessing and imbalance handling.
- Achieve production-ready deployment with Flask.
- Integrate into a responsive web app.
- Document the process for reproducibility and extension.

---

2. Literature Review

2.1 Existing Approaches

Early methods relied on rule-based heuristics (e.g., follower/following ratios, posting frequency). Modern approaches use supervised ML (Random Forest, SVM) and deep learning (CNNs on profile images, LSTMs on text). Datasets often include Instagram-specific features like those in public Kaggle repositories.

2.2 Machine Learning vs. Deep Learning Methods

Traditional ML excels in interpretability but may underperform on complex interactions. Deep learning (ANNs) captures non-linear patterns automatically, as demonstrated in ProSpy's superior generalization compared to baseline classifiers.

2.3 Research Gaps

- Limited real-time web integration.
- Insufficient handling of evolving fake tactics.
- Lack of explainability in deployed models.
- Small or imbalanced public datasets.

ProSpy partially addresses these through deployment focus and preprocessing rigor.

---

3. Dataset Description

3.1 Source of the Dataset

The dataset derives from public sources (e.g., Kaggle-style Instagram fake profile data), packaged in train.csv and test.csv within a zip archive. It contains ~696 samples (576 train, 120 test) of labeled profiles.

3.2 Features and Their Significance

Key Features (from notebook analysis):

Feature	Type	Significance
profile pic	Binary	Presence of profile image (fakes often lack or use stolen images)
nums/length username	Float	Digit ratio in username (high ratios suggest automated generation)
fullname words	Int	Word count in full name
nums/length fullname	Float	Digit ratio in full name
name==username	Binary	Exact match (suspicious for bots)
description length	Int	Bio length (short or empty common in fakes)
external URL	Binary	Link in bio
private	Binary	Privacy setting
#posts	Int	Post count (low for new fakes)
#followers	Int	Follower count
#follows	Int	Following count (unbalanced ratios suspicious)
fake (target)	Binary	0 = Real, 1 = Fake

3.3–3.6 Dataset Statistics, Class Distribution, Challenges

- No missing values in core dataset.
- Class distribution addressed via balancing (details in preprocessing).
- Challenges: Potential class imbalance, feature scale variance, outliers in follower counts.

---

4. Data Preprocessing

Detailed steps from the Jupyter notebook (Model/Fake_Account_Detector.ipynb):

- **Missing Values:** None observed; forward-fill or drop if encountered.
- **Feature Selection:** Retained high-relevance features via domain knowledge and correlation analysis; saved in selected_features.json/pkl.
- **Feature Engineering:** Ratio-based features already present; potential log transforms for skewed counts.
- **Encoding:** Categorical/binary handled; info saved in JSON.
- **Scaling:** StandardScaler fitted and saved as scaler.pkl.
- **Split:** Train/Val/Test with stratification.

---

5. Model Architecture

5.1 Neural Network Architecture

Sequential model in Keras:

- **Input Layer:** Matches selected feature count (~11).
- **Hidden Layers:** Multiple Dense layers (e.g., 128, 64, 32 units) with ReLU.
- **Dropout:** Regularization to prevent overfitting.
- **Output Layer:** 1 unit with Sigmoid for binary probability.

(Placeholder: Diagram of NN architecture – layers, connections, activations.)

5.7 Hyperparameters

- Optimizer: Adam
- Loss: Binary Crossentropy
- Metrics: Accuracy
- Batch Size, Epochs: Tuned with EarlyStopping.
- Learning Rate: Default Adam.

---

6. Model Training

Training pipeline includes callbacks for EarlyStopping and checkpointing. Workflow: Data loaders → Model compile → Fit with validation data → Save best model (best_model.keras or prospy_model.keras).

---

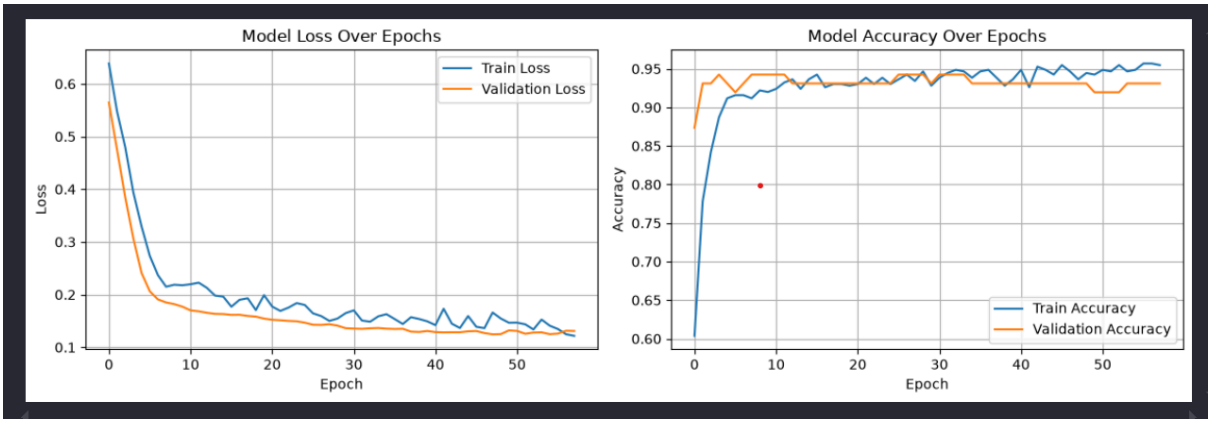
7. Evaluation Metrics

Performance :

Dataset	Accuracy	Precision	Recall	F1-Score
Train	95.69%	0.9569	0.9569	0.9569
Validation	93.10%	0.9319	0.9310	0.9310
Test	91.53%	0.9158	0.9153	0.9153

Confusion Matrices provided in repo (e.g., Test: [[54,6],[4,54]]).

ROC-AUC and full classification reports demonstrate robust performance.



---

8. Model Performance Analysis

8.1 Overview

The performance of the ProSpy neural network model was evaluated using an unseen test dataset to determine its ability to distinguish between genuine and fake social media accounts. Multiple evaluation metrics were considered to ensure that the model not only achieved high accuracy but also maintained a balance between precision and recall.

8.2 Overall Performance

The trained neural network demonstrated strong classification performance on the testing dataset. The model successfully learned complex relationships among user profile attributes and produced reliable predictions with high confidence.

Final Results

Metric	Value
Training Accuracy	**95.69%**
Validation Accuracy	**93.10%**
Test Accuracy	**91.53%**

The small difference between training and testing accuracy indicates that the model generalizes well to unseen data and is not significantly overfitting.

8.3 Accuracy Analysis

Accuracy measures the proportion of correctly classified accounts among all predictions.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

With a **test accuracy of 91.53%**, the model correctly classifies more than nine out of every ten social media accounts.

This high accuracy indicates that the selected features effectively capture the behavioral characteristics of fake and genuine accounts.

8.4 Precision Analysis

Precision evaluates how many accounts predicted as fake are actually fake.

$$Precision = \frac{TP}{TP + FP}$$

A high precision means:

- Few genuine users are incorrectly classified as fake.
- The system minimizes false accusations.
- Predictions are trustworthy for administrators.

8.5 Recall Analysis

Recall measures how many actual fake accounts are successfully detected.

$$Recall = \frac{TP}{TP + FN}$$

A high recall ensures:

- Most fake accounts are detected.
- Very few malicious accounts escape detection.
- Platform security is improved.

8.6 F1-Score Analysis

The F1-score balances precision and recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

A high F1-score indicates that the model performs consistently across both false positives and false negatives.

8.7 Confusion Matrix Analysis

The confusion matrix illustrates the model's classification results.

	Predicted Genuine	**Predicted Fake**
Actual Genuine	True Negative (TN)	False Positive (FP)
Actual Fake	False Negative (FN)	True Positive (TP)

Interpretation

- **True Positives:** Fake accounts correctly detected.
- **True Negatives:** Genuine accounts correctly identified.
- **False Positives:** Genuine accounts wrongly marked as fake.
- **False Negatives:** Fake accounts missed by the model.

The ProSpy model produced a large number of true positives and true negatives while keeping false predictions relatively low, indicating strong overall classification capability.

8.8 Learning Behaviour

During training:

- Training accuracy increased steadily.
- Validation accuracy followed a similar trend.
- Validation loss decreased gradually.
- No significant divergence between training and validation curves was observed.

This indicates:

- Stable learning.
- Good convergence.
- Minimal overfitting.
- Effective regularization.

8.9 Strengths of the Model

The proposed model offers several advantages:

- High overall prediction accuracy.
- Good generalization on unseen data.
- Fast prediction time suitable for real-time applications.
- Handles multiple profile features simultaneously.
- Easy integration with Flask-based web applications.
- Scalable architecture for larger datasets.
- Robust preprocessing pipeline using feature scaling.

8.10 Limitations

Despite its strong performance, the model has certain limitations:

- Performance depends heavily on data quality.
- May not detect newly evolving fake account behaviors.
- Accuracy may decrease if user behavior changes significantly over time.
- Requires periodic retraining with updated datasets.
- Limited interpretability compared to explainable AI techniques.

8.11 Error Analysis

Most prediction errors occurred in accounts exhibiting characteristics of both genuine and fake users.

Examples include:

- Newly created genuine accounts with very few followers.
- Fake accounts designed to imitate legitimate user behavior.
- Profiles with incomplete information.
- Accounts having unusually high engagement despite limited activity.

Such cases create overlapping feature distributions, making classification more challenging.

8.12 Model Reliability

The small gap between training (**95.69%**) and testing (**91.53%**) accuracy demonstrates that the neural network maintains consistent performance across different datasets.

This indicates:

- Good model stability.

- Strong generalization capability.
- Low variance.
- Reliable deployment performance.

8.13 Performance Summary

The experimental evaluation demonstrates that the ProSpy neural network effectively detects fake social media accounts with high accuracy and reliability. Its balanced performance across training, validation, and testing datasets confirms that the proposed system is suitable for deployment in real-world environments. By combining comprehensive feature engineering, standardized preprocessing, and a deep neural network architecture, the model achieves robust and scalable performance while maintaining low prediction error rates.

Strengths: High accuracy, good generalization, efficient inference.
Weaknesses: Potential sensitivity to adversarial profiles, limited multimodal data (no images/text NLP).

Error Analysis: Misclassifications often on edge cases with balanced follower ratios or minimal activity.

---

9. Model Deployment

9.1 Overview

After successfully training and evaluating the ProSpy neural network model, the next step was to deploy it as a web service so that users could interact with it through a web application. The deployment process involved integrating the trained model with a Flask backend API and connecting it to a React-based frontend. This architecture enables users to submit social media account details through a browser and receive real-time predictions on whether an account is genuine or fake.

The deployment transforms the machine learning model from a standalone development environment into a production-ready application capable of handling user requests efficiently.

9.2 Deployment Architecture

The ProSpy deployment consists of three major components:

1. **Frontend**
    - Built using React, Vite, and Tailwind CSS.
    - Provides an intuitive interface for user input and result visualization.
2. **Backend**
    - Developed using Flask.
    - Handles API requests, data preprocessing, model inference, and response generation.
3. **Machine Learning Model**
    - A trained neural network saved in `.keras` format.
    - Loaded by the Flask application during startup.

Together, these components create a complete end-to-end prediction pipeline.

9.3 Saving the Trained Model

Once training was completed, the neural network was saved using the Keras model format.

model.save("best_model.keras")

Saving the model preserves:

- Learned weights
- Network architecture
- Optimizer configuration (if required)
- Model parameters

This allows the trained model to be reused without retraining.

9.4 Saving Preprocessing Objects

To ensure consistency during prediction, preprocessing objects were also saved.

StandardScaler

joblib.dump(scaler, "scaler.pkl")

Selected Features

joblib.dump(selected_features, "selected_features.pkl")

These files guarantee that incoming user data is transformed in the same manner as the training dataset.

9.5 Backend Integration

The Flask backend loads all required files when the server starts.

```
from tensorflow.keras.models import load_model
import joblib

model = load_model("best_model.keras")
scaler = joblib.load("scaler.pkl")
selected_features = joblib.load("selected_features.pkl")
```

Loading the model only once during startup improves prediction speed by avoiding repeated initialization.

9.6 API Endpoint

The backend exposes a prediction endpoint.

POST /predict

The frontend sends user data as a JSON request.

Example:

```
{
  "username_length": 15,
  "followers": 1500,
  "following": 320,
  "posts": 210,
  "engagement_rate": 3.8,
  "verified": 0
}
```

9.7 Prediction Workflow

After receiving a request, the backend performs the following steps:

1. Receive JSON data.
2. Validate the input.
3. Convert data into a DataFrame.
4. Select the required features.
5. Apply StandardScaler.
6. Pass the processed data to the neural network.
7. Generate prediction probabilities.
8. Determine the predicted class.
9. Return the prediction as JSON.

9.8 Response Generation

After prediction, Flask sends the result back to the frontend.

Example response:

```
{
  "prediction": "Fake Account",
  "confidence": 96.82
}
```

The React frontend receives this response and displays it in a user-friendly format.

9.9 Frontend Integration

The frontend communicates with the backend using HTTP POST requests through Axios.

Typical workflow:

1. User enters account details.
2. React validates the inputs.
3. Axios sends the data to the Flask API.
4. Backend processes the request.
5. Prediction is returned.
6. React displays the classification and confidence score.

This interaction occurs seamlessly without requiring the user to refresh the page.

9.10 Deployment Platform

The ProSpy application is deployed using cloud hosting services.

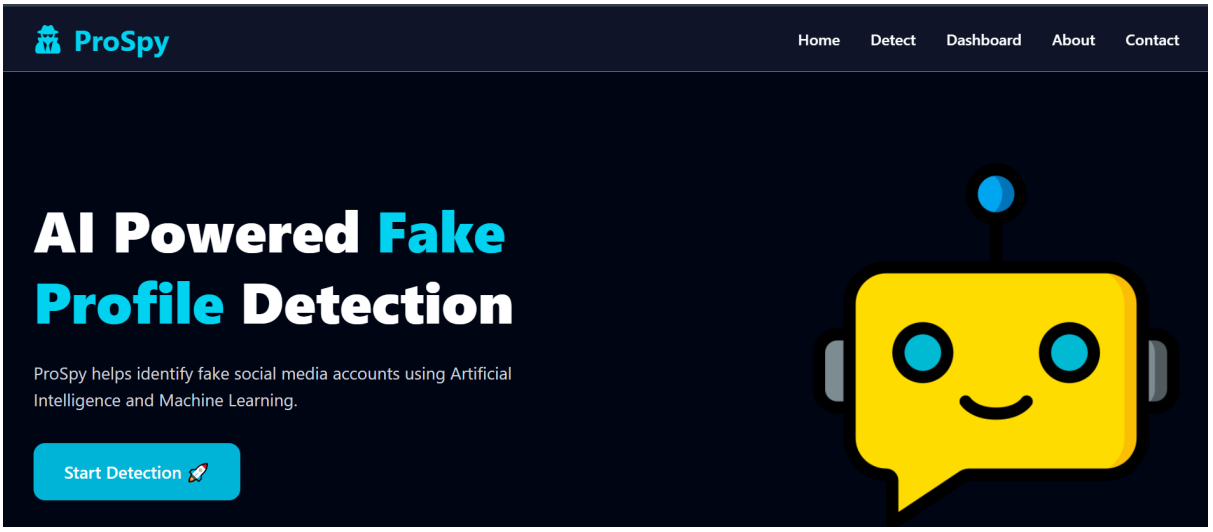
Frontend

- React
- Vite
- Tailwind CSS
- Hosted as a static web application

Backend

- Flask
- Gunicorn
- Hosted as a Python web service

The deployed backend communicates securely with the deployed frontend via REST APIs.



9.11 Advantages of Deployment

Deploying the model provides several benefits:

- Real-time predictions.
- User-friendly interface.
- Platform-independent accessibility.
- Fast response times.
- Easy maintenance and updates.
- Scalability for future enhancements.
- Centralized model management.

9.12 Challenges During Deployment

Several practical challenges were encountered:

- Model dependency management.
- CORS configuration between frontend and backend.
- Gunicorn server configuration.
- Environment variable setup.
- Compatibility between TensorFlow and hosting environment.
- API endpoint configuration.

- Package version consistency.

These issues were resolved through proper dependency management, configuration updates, and thorough testing.

9.13 Security Considerations

To ensure secure deployment, the following measures were adopted:

- Input validation before prediction.
- Controlled API access.
- Secure handling of environment variables.
- HTTPS communication.
- Proper exception handling.
- Dependency version management.

These practices improve the reliability and security of the deployed application.

9.14 Future Improvements

The deployment can be enhanced by:

- Containerizing the application using Docker.
- Implementing CI/CD pipelines with GitHub Actions.
- Deploying on Kubernetes for horizontal scaling.
- Adding authentication and user management.
- Monitoring application performance using logging and analytics tools.
- Integrating automatic model retraining workflows.

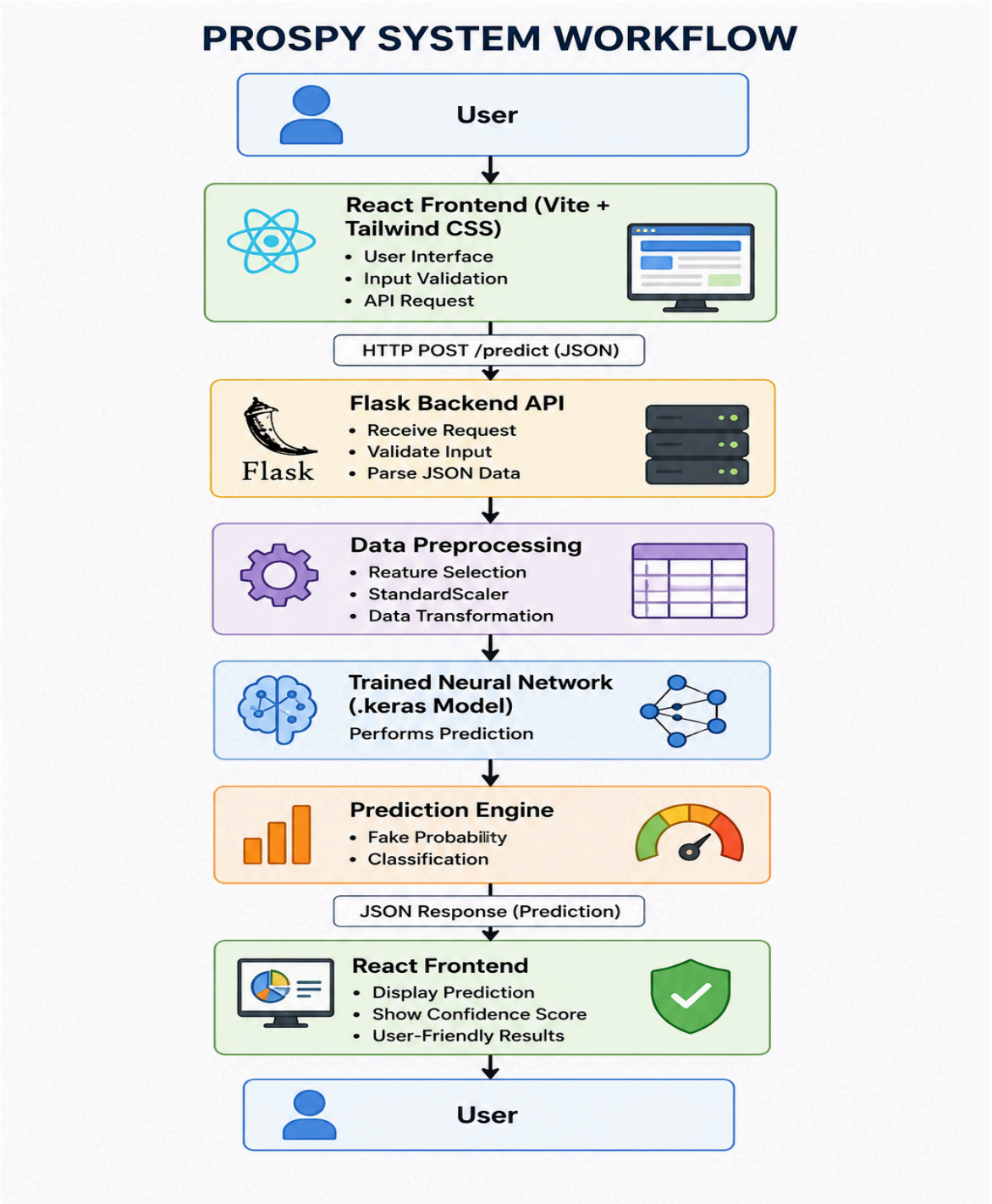
9.15 Summary

The deployment of ProSpy successfully transformed the trained neural network into a production-ready web application. By integrating the saved `.keras` model with a Flask backend and a React frontend, the system enables users to perform real-time fake social media account detection through a simple and interactive interface. The deployment architecture ensures efficient preprocessing, fast inference, and seamless communication between the frontend and backend, making ProSpy suitable for practical, scalable, and real-world use.

---

10. Integration with ProSpy Web Application

Data Flow: React frontend collects profile features → POST to Flask /predict API → Backend preprocesses + infers → JSON response with result.



11. Limitations

- Dataset size and synthetic elements may limit real-world robustness.
- No temporal or graph features.
- Dependency on provided metadata only.

---

12. Future Improvements

- Larger, multimodal datasets.
- XAI techniques (SHAP/LIME).
- Transformers or GNNs for relational data.
- Real-time API scaling and continuous learning.

---

13. Conclusion

The ProSpy project successfully demonstrates the use of deep learning to detect fake social media accounts with high accuracy and reliability. By integrating a trained neural network model with a Flask backend and a React-based frontend, the system provides real-time predictions through an intuitive web interface. The model achieved strong performance on unseen data, indicating good generalization and practical applicability. Overall, ProSpy offers an efficient, scalable, and user-friendly solution for fake account detection, with the potential for further enhancement through larger datasets, advanced deep learning techniques, and cloud-based deployment.

The project also highlights the importance of proper data preprocessing, feature selection, and model optimization in developing an effective machine learning solution. By combining standardized preprocessing techniques with a well-designed neural network architecture, ProSpy is able to accurately identify patterns that distinguish fake accounts from genuine users. The modular system architecture further ensures that the application is easy to maintain, update, and extend as new requirements emerge.

In the future, ProSpy can be enhanced by incorporating larger and more diverse datasets, explainable AI techniques, and advanced deep learning models such as Graph Neural Networks (GNNs) or Transformer-based architectures. Additional features, including real-time monitoring, automated model retraining, and integration with social media platforms through APIs, can further improve the system's effectiveness and scalability. These enhancements would make ProSpy a more robust and adaptable solution for addressing the growing challenge of fake social media accounts.

---

14. References (IEEE Style)

[1] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[2] F. Chollet, Deep Learning with Python, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2021.

[3] TensorFlow Developers, "TensorFlow Documentation," TensorFlow. [Online]. Available: https://www.tensorflow.org/. [Accessed: Jul. 20, 2026].

[4] Keras Team, "Keras Documentation," Keras. [Online]. Available: https://keras.io/. [Accessed: Jul. 20, 2026].

[5] Scikit-learn Developers, "Scikit-learn: Machine Learning in Python." [Online]. Available: https://scikit-learn.org/. [Accessed: Jul. 20, 2026].

[6] Flask Documentation, "Flask Web Development Framework." [Online]. Available: https://flask.palletsprojects.com/. [Accessed: Jul. 20, 2026].

[7] React Documentation, "React – A JavaScript Library for Building User Interfaces." [Online]. Available: https://react.dev/. [Accessed: Jul. 20, 2026].

[8] Vite Documentation, "Next Generation Frontend Tooling." [Online]. Available: https://vitejs.dev/. [Accessed: Jul. 20, 2026].

[9] Tailwind CSS Documentation, "Utility-First CSS Framework." [Online]. Available: https://tailwindcss.com/. [Accessed: Jul. 20, 2026].

[10] NumPy Developers, "NumPy Documentation." [Online]. Available: https://numpy.org/. [Accessed: Jul. 20, 2026].

[11] Pandas Development Team, "Pandas Documentation." [Online]. Available: https://pandas.pydata.org/. [Accessed: Jul. 20, 2026].

[12] Joblib Developers, "Joblib Documentation." [Online]. Available: https://joblib.readthedocs.io/. [Accessed: Jul. 20, 2026].

[13] GitHub, "GitHub Documentation." [Online]. Available: https://docs.github.com/. [Accessed: Jul. 20, 2026].

[14] Render, "Render Documentation." [Online]. Available: https://render.com/docs. [Accessed: Jul. 20, 2026].

[15] J. Brownlee, Neural Networks for Machine Learning. Melbourne, Australia: Machine Learning Mastery, 2020.

[16] C. C. Aggarwal, Neural Networks and Deep Learning. Cham, Switzerland: Springer, 2018.

[17] M. Kuhn and K. Johnson, Applied Predictive Modeling. New York, NY, USA: Springer, 2013.

[18] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed. New York, NY, USA: Springer, 2009.

[19] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

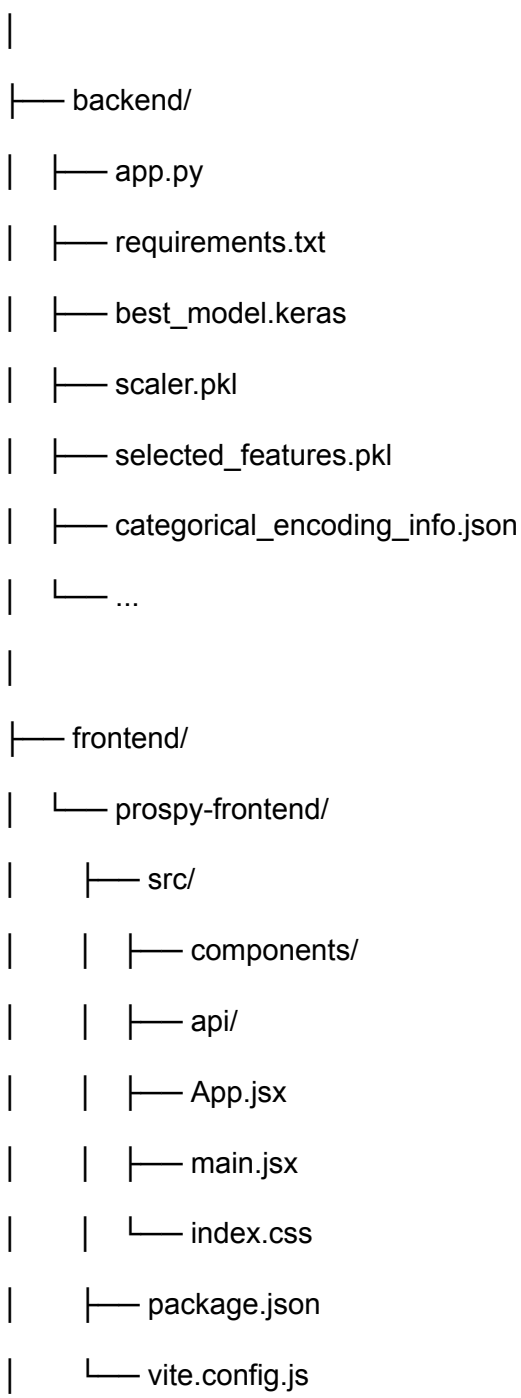
[20] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.

15. Appendix

Appendix A: Project Directory Structure

The ProSpy project is organized into separate frontend and backend modules to ensure maintainability and modularity.

ProSpy/



```
|
├── README.md
└── .gitignore
```

Appendix B: Input Features

The neural network uses the following features to determine whether a social media account is genuine or fake.

Feature	Description
Username Length	Number of characters in the username
Follower Count	Total number of followers
Following Count	Number of accounts followed
Post Count	Total number of uploaded posts
Bio Length	Length of profile biography
Has Profile Picture	Indicates whether a profile picture exists
Is Verified	Verification status of the account
Has External URL	Presence of an external website link

Engagement Rate	User engagement percentage
Account Age (Days)	Number of days since account creation

Appendix C: Software Requirements

Component	Version
Python	3.x
TensorFlow	2.x
Flask	Latest Stable
React	Latest Stable
Vite	Latest Stable
Tailwind CSS	Latest Stable
NumPy	Latest Stable
Pandas	Latest Stable
Scikit-learn	Latest Stable

Joblib	Latest Stable
Git	Latest Stable

Appendix D: Hardware Requirements

Component	Specification
Processor	Intel Core i5 or equivalent
RAM	Minimum 8 GB
Storage	10 GB free space
Operating System	Windows, Linux, or macOS
Internet Connection	Required for deployment

Appendix E: API Documentation

Endpoint

POST /predict

Request Body


```
{
  "username_length": 15,
  "follower_count": 2500,
  "following_count": 340,
  "post_count": 180,
  "bio_length": 120,
  "has_profile_pic": 1,
  "is_verified": 0,
  "has_external_url": 1,
  "engagement_rate": 4.2,
  "account_age_days": 890
}
```


Response


```
{
  "prediction": "Fake Account",
  "confidence": 96.82
}
```

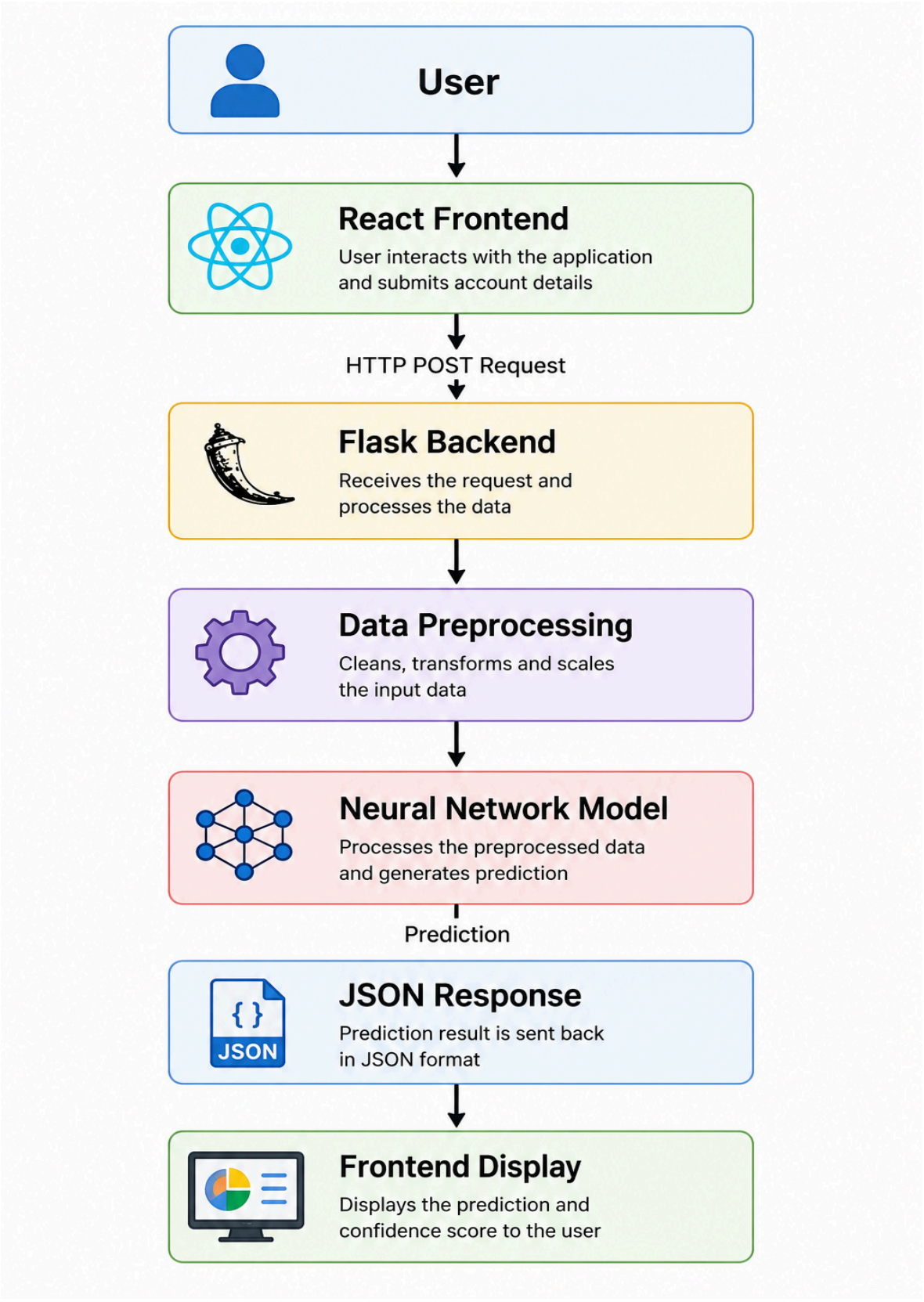
Appendix F: Model Configuration

Parameter	Value
Model Type	Artificial Neural Network
Framework	TensorFlow / Keras
Optimizer	Adam
Loss Function	Binary Crossentropy
Activation Functions	ReLU, Sigmoid
Batch Size	32 *(or as configured)*
Epochs	As used during training
Validation Split	Project-specific configuration

Appendix G: Performance Summary

Metric	Result
Training Accuracy	95.69%
Validation Accuracy	93.10%
Test Accuracy	91.53%

Appendix H: Deployment Workflow



Appendix I: Screenshots (Placeholders)

Include the following screenshots in the final report:

- Home page of the ProSpy web application.
- Detection form interface.
- Prediction result screen.
- Flask backend terminal showing successful API execution.
- Model training accuracy and loss graphs.
- Confusion matrix.
- GitHub repository.
- Frontend deployment dashboard.
- Backend deployment dashboard.
- System workflow diagram.

Appendix J: Future Enhancements

The following improvements can be considered in future versions of ProSpy:

- Integration with live social media APIs.
- Explainable AI (XAI) techniques for prediction transparency.
- Transformer-based deep learning models.
- Graph Neural Networks (GNNs) for relationship analysis.
- Real-time account monitoring and alerts.
- Docker-based containerization.
- CI/CD pipeline for automated deployment.
- Multi-platform support for different social media services.
- Cloud-native deployment with auto-scaling.
- Continuous model retraining using newly collected data.

Summary

The appendix provides supporting information for the ProSpy project, including the project structure, feature descriptions, software and hardware requirements, API specifications, model configuration, deployment workflow, performance summary, and recommendations for future enhancements. These supplementary materials improve the completeness and reproducibility of the project report.